Configuration

GemStone Database Explorer is configured entirely through environment variables, read at startup by `gemstone-py`.

The canonical connection variable names are `GS_USERNAME`, `GS_PASSWORD`, `GS_STONE`, `GS_HOST`, and `GS_NETLDI`. Shorter aliases such as `GS_USER` or longer aliases such as `GS_NETLDI_NAME_OR_PORT` are not primary configuration keys for this app.

Required variables

Variable	Description
`GEMSTONE`	Absolute path to your GemStone installation directory
`GS_USERNAME`	GemStone user to log in as (e.g. `DataCurator`)
`GS_PASSWORD`	Password for that user

Optional variables

Variable	Description	Default
`GS_STONE`	Stone name to connect to	`gs64stone`
`GS_STONE_NAME`	Compatibility alias for `GS_STONE` in this app	unset
`GS_HOST`	Host running the stone/netldi	`localhost`
`GS_NETLDI`	NetLDI service name or port	`netldi`
`GS_GEM_SERVICE`	Gem service name	`gemnetobject`
`GS_LIB_PATH`	Explicit path to the GCI shared library	auto-discover

If your running Stone is named `seaside`, export `GS_STONE=seaside` before starting the app. `GS_STONE_NAME` is accepted here as a compatibility alias, but the underlying `gemstone-py` client natively reads `GS_STONE`.

The explorer taskbar `Connection` window and `GET /connection/preflight` surface the effective target, the local `gslist -lcv` probe when available, and suggested fixes for common mistakes such as leaving the default `gs64stone` configured when the running Stone is actually named `seaside`.

Example

```
export GEMSTONE=/opt/gemstone/GemStone64Bit3.7.5-arm64.Darwin
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
export GS_HOST=localhost
export GS_NETLDI=50377
export GS_STONE=seaside

.venv/bin/python-gemstone-database-explorer
```

GCI library path

The GemStone GCI shared library must be on the dynamic linker path:

```
# macOS
export DYLD_LIBRARY_PATH=$GEMSTONE/lib:$DYLD_LIBRARY_PATH

# Linux
export LD_LIBRARY_PATH=$GEMSTONE/lib:$LD_LIBRARY_PATH
```

Session management

The Flask app uses a small broker over `GemStoneSession` objects rather than naive per-request login/logout.

- requests are routed into channel families, either from explicit `X-GS-Channel` headers or by route defaults
- managed sessions are created lazily and reused within those channel families
- read-only requests still abort on exit to release transaction state cleanly
- write flows keep their managed session alive so debugger state and browser context can survive across requests

GemStone/GCI access is still serialized behind a process-global lock, so this improves isolation and recovery but does not make a single Flask process fully parallel for GemStone work.